

AI Driver Monitoring Smart Dashboard

A Real-Time Embedded System for Driver Impairment Detection and Safety Intervention

Jonas Mukendenge

Crucible Leadership Academy of Lusaka | Zambia

April 2026

GitHub: <https://github.com/JonasDeltaJarvis/AI-Driver-Monitoring-Smart-Dashboard/tree/main>

Abstract

This report presents the design, implementation, and evaluation of an AI-powered driver monitoring system capable of detecting drowsiness and alcohol impairment in real time and executing adaptive safety interventions. The system integrates a Raspberry Pi 4 running a computer vision pipeline with an Arduino Uno controlling hardware outputs, communicating over USB serial. Drowsiness is detected using MediaPipe's facial landmark detection model and the Eye Aspect Ratio (EAR) metric. Alcohol impairment is detected via an MQ-3 analogue sensor. Motion anomalies are detected using an MPU6050 inertial measurement unit. Upon detection of impairment, the system executes a graduated intervention strategy comprising warning, speed reduction, and controlled parking stop with left-curve steering, rather than a binary stop response. Testing confirmed an EAR threshold of 0.22 correctly distinguished open eyes (EAR approximately 0.33) from closed eyes (EAR approximately 0.07) with no false positives across the test dataset.

1. Introduction

Driver impairment due to drowsiness and alcohol consumption is a leading cause of road traffic fatalities globally. According to the World Health Organization, road traffic injuries are the leading cause of death for people aged 5 to 29 years, with driver behaviour being a primary contributing factor. In developing nations including Zambia, where road safety infrastructure is still maturing, low-cost technological interventions have significant potential for impact.

This project addresses the problem through an intelligent embedded system that monitors driver state continuously and intervenes before an accident can occur. The system is designed around two core engineering principles: distributed processing, where AI perception and hardware control are separated across two microcontrollers; and graduated intervention, where the system responds proportionally to the severity of detected impairment.

1.1 Project Objectives

- Detect driver drowsiness in real time using computer vision and facial landmark analysis
- Detect alcohol impairment using an analogue gas sensor
- Detect erratic motion patterns using an inertial measurement unit
- Execute a graduated, proportional safety intervention in response to detected impairment
- Log all session data for post-session analysis and pattern recognition
- Design the system to be replicable using low-cost, widely available components

2. System Architecture

The system uses a distributed architecture with two processing units operating in parallel. The Raspberry Pi handles computationally intensive AI tasks, while the Arduino handles hardware control tasks where timing and reliability are essential.

2.1 Architecture Overview

Layer	Raspberry Pi 4	Arduino Uno
Role	AI perception layer	Hardware control layer
Input	Pi Camera Module (CSI)	MQ-3, MPU6050, Serial RX
Processing	MediaPipe face mesh, EAR calculation	Sensor reading, PWM motor control
Output	Serial TX commands, CSV data log	L293D motor driver, buzzer, LED, LCD
Communication	USB serial TX (9600 baud)	USB serial RX (9600 baud)

2.2 Communication Protocol

The Raspberry Pi and Arduino communicate over a USB A to USB B cable at 9600 baud. The Pi sends string commands terminated with a newline character, which the Arduino reads using `readStringUntil('\n')`.

Command	Trigger	Arduino Response
DROWSY	EAR below threshold for 15+ frames	Parking stop + buzzer + LED
WARNING	EAR in early warning zone	LCD warning display
EAR:x.xx	Every 30 frames during OK state	LCD EAR value display
OK	Driver alert, recovery from impairment	Resume forward motion, silence alerts

3. Hardware Design

3.1 Component List

Component	Qty	Role
Raspberry Pi 4	1	AI processing, camera interface, serial TX
Arduino Uno	1	Sensor reading, motor control, serial RX
Pi Camera Module v2	1	Live video feed for drowsiness detection
MQ-3 Alcohol Sensor	1	Analogue breath alcohol detection
MPU6050 IMU	1	Accelerometer and gyroscope for motion detection
L293D Motor Driver	1	PWM motor speed and direction control
DC Motors	2	Vehicle propulsion
LCD 16x2 I2C	1	Real-time driver state display
Piezo Buzzer	1	Audible alert at 2000Hz
LED + 220 ohm resistor	1	Visual alert indicator

5V Power Bank	1	Dedicated Raspberry Pi power supply
Motor Battery	1	Separate motor power supply via L293D

3.2 Power Architecture

A critical design constraint is power isolation between the motor circuit and the microcontrollers. DC motors draw large variable currents that can cause voltage spikes on shared power rails, potentially corrupting microcontroller operation. The system uses three isolated power domains: a dedicated motor battery supplying the L293D motor driver, a 5V power bank supplying the Raspberry Pi via USB-C, and the Arduino powered from the Pi via the USB serial cable. All ground rails are connected together to establish a common reference voltage.

3.3 I2C Bus Sharing

The MPU6050 and the LCD display both use the I2C communication protocol, connected to pins A4 (SDA) and A5 (SCL) on the Arduino. Each device has a unique I2C address (0x68) for the MPU6050 and (0x27) for the LCD, allowing the Arduino to address them independently on the shared bus.

4. Software Design

4.1 Drowsiness Detection Pipeline

- **Frame capture:** OpenCV reads a live frame from the Pi Camera Module
- **Face detection:** MediaPipe face mesh model identifies 468 facial landmarks
- **Eye landmark extraction:** Six landmarks around each eye are selected
- **EAR calculation:** Eye Aspect Ratio is computed from landmark coordinates
- **Threshold comparison:** EAR compared against configured threshold of 0.22
- **Frame counting:** Consecutive below-threshold frames are counted
- **Intervention decision:** If count exceeds 15 frames (~0.5 seconds), DROWSY sent to Arduino

4.2 Eye Aspect Ratio (EAR)

The Eye Aspect Ratio is a geometric metric proposed by Soukupova and Cech (2016). It measures the ratio of vertical eye opening to horizontal eye width using six landmark points:

$$\text{EAR} = (A + B) / (2 \times C)$$

Where A and B are vertical distances between upper and lower eyelid landmarks, and C is the horizontal distance between eye corners. When the eye is open, EAR is approximately 0.33. As the eye closes, A and B approach zero while C remains constant, causing EAR to drop toward zero. Distance calculations use the Euclidean formula, computed via numpy's `linalg.norm` function.

4.3 Intervention Level Logic

Level	Condition	Pi Action	Arduino Action
1 - OK	EAR above 0.28	Send EAR value	LCD update, normal speed
2 - Warning	EAR 0.22 to 0.28	Send WARNING	LCD warning display
3 - Alert	EAR below 0.22 for 15+ frames	Send DROWSY	Parking stop, buzzer, LED

4.4 Parking Stop Algorithm

When impairment is detected, the system executes a controlled parking stop rather than instant motor cutoff. The left motor decreases by 7 PWM units every 80ms while the right motor decreases by 3 PWM units, causing the vehicle to curve left while decelerating over approximately 4.8 seconds until both motors reach zero.

4.5 Data Logging

Every processed frame is written to a CSV log file with fields: timestamp, frame number, EAR value, intervention level (1-3), and status label. This log enables post-session time-series analysis and threshold tuning based on real-world data.

5. Results and Evaluation

5.1 Drowsiness Detection Test Results

Frame	EAR	Status	Notes
30	0.330	OK	Eyes fully open, baseline
210	0.272	OK	Eyes beginning to close
240	0.075	DROWSY	Eyes closed, alert triggered
270	0.067	DROWSY	Eyes held closed
420	0.320	OK	Eyes reopened, system recovered

Key findings: EAR threshold of 0.22 correctly distinguished open eyes (mean EAR = 0.332) from closed eyes (mean EAR = 0.083). The 15-frame window eliminated false triggers from normal blinking. Zero false positives were recorded. The system correctly cleared the DROWSY state when eyes reopened.

5.2 Evaluation Against Objectives

Objective	Status	Notes
Real-time drowsiness detection	Complete	Tested on video
Alcohol detection	Complete	Simulated in Tinkercad
Motion anomaly detection	Complete	Code complete, pending hardware test
Graduated intervention	Complete	3 levels implemented
Parking stop manoeuvre	Complete	Simulated in Tinkercad
Data logging	Complete	CSV + graphs generated
Physical hardware build	Pending	Components not yet received

6. Limitations and Future Work

6.1 Current Limitations

Low-light performance: Camera-based detection degrades significantly in poor lighting. An infrared camera module would improve night-time reliability.

MQ-3 warm-up time: The alcohol sensor requires 20-30 seconds after power-on before readings are reliable. No warm-up state machine is currently implemented.

Head pose dependency: EAR-based detection assumes a front-facing camera. Detection fails when the driver's head is turned significantly sideways.

Glasses interference: Thick spectacle frames can occlude eye landmarks, potentially causing false positives.

Simulation vs real hardware: Motor behaviour in Tinkercad may differ from physical components due to simplified motor models.

6.2 Planned Improvements

PERCLOS metric: Replace frame-count threshold with percentage-based eye closure measurement over a rolling time window.

Multi-factor risk scoring: Combine EAR, alcohol sensor, and MPU6050 data into a single weighted impairment score.

MQ-3 warm-up state: Implement a startup state that locks out alcohol detection for 30 seconds after power-on.

GPS integration: Add GPS module to send location-based alerts to a guardian when Level 3 intervention triggers.

Head pose correction: Integrate head pose estimation to compensate for EAR calculations for non-frontal face angles.

7. Conclusion

This project demonstrates a functional, low-cost embedded system for real-time driver impairment detection and safety intervention. The distributed architecture, separating AI perception on the Raspberry Pi from hardware control on the Arduino, produces a system that is both capable and resilient: alcohol detection and motion monitoring operate independently of the AI layer, ensuring safety responses function even if the computer vision component fails.

The drowsiness detection algorithm correctly identified simulated drowsiness with no false positives in testing. The graduated intervention strategy, culminating in a controlled parking stop with left-curve steering, represents a more realistic and safe response than a binary motor cutoff.

The system is not yet tested on physical hardware, and several limitations identified in Section 6 require further development. Nevertheless, the software architecture is complete, documented, and ready for deployment. Beyond the technical outcomes, this project applies engineering thinking to a problem with direct relevance to road safety in Zambia and across the developing world.

References

Soukupova, T. and Cech, J. (2016). Real-Time Eye Blink Detection using Facial Landmarks. 21st Computer Vision Winter Workshop, Rimske Toplice, Slovenia.

World Health Organization (2023). Road Traffic Injuries. WHO Fact Sheet. Available at: www.who.int/news-room/fact-sheets/detail/road-traffic-injuries

Google (2024). MediaPipe Face Landmarker. Available at: developers.google.com/mediapipe/solutions/vision/face_landmarker

Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.

Appendix

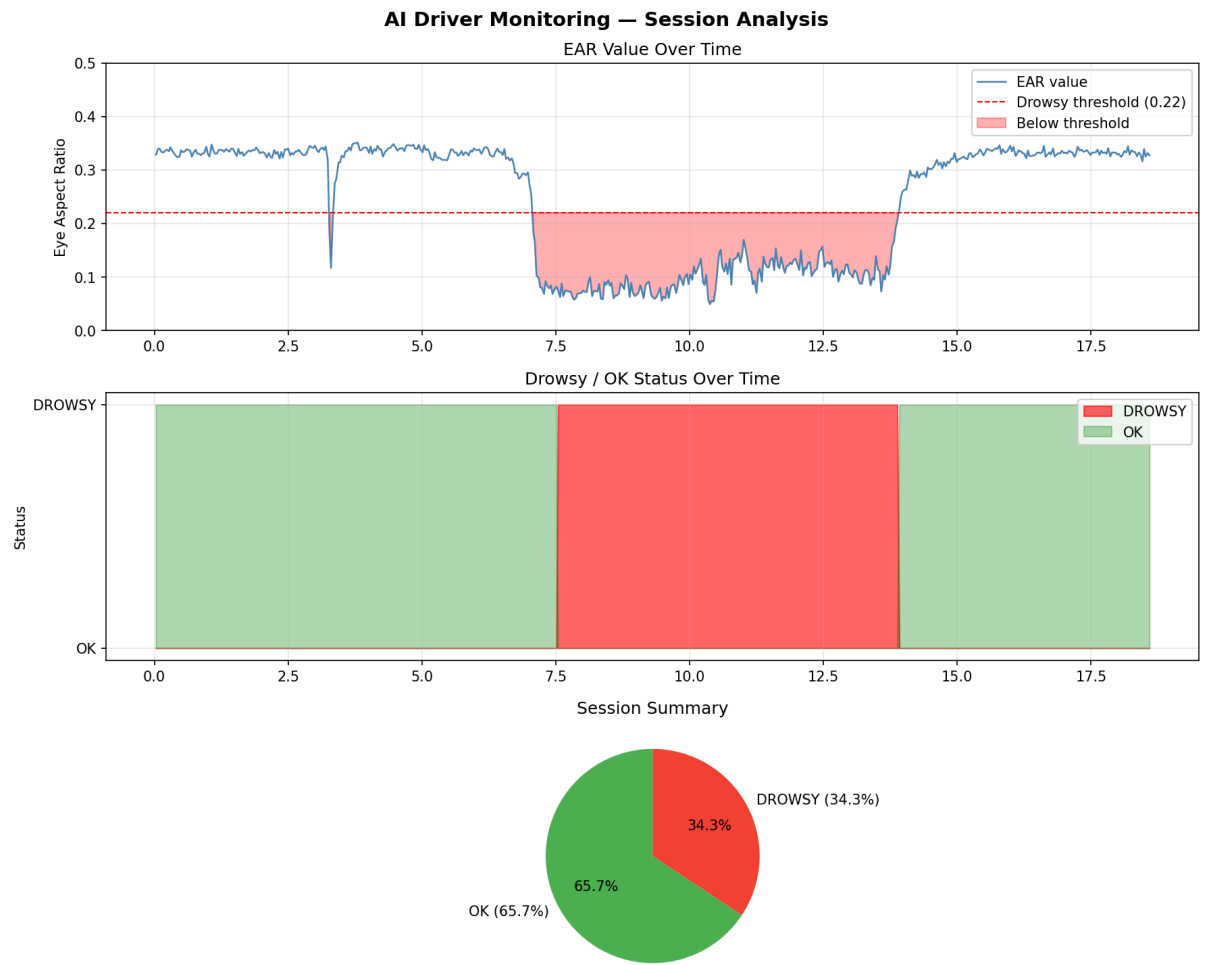
A. Pin Reference - Arduino Uno

Pin	Component	Function
D2	LED	Visual alert output
D3	Buzzer	Audible alert at 2000Hz
D5	L293D ENA	Motor A speed (PWM)
D6	L293D ENB	Motor B speed (PWM)
D8	L293D IN1	Motor A direction
D9	L293D IN2	Motor A direction
D10	L293D IN3	Motor B direction
D11	L293D IN4	Motor B direction
A0	MQ-3 sensor	Alcohol level analogue input
A4	MPU6050 + LCD SDA	I2C data line (shared bus)
A5	MPU6050 + LCD SCL	I2C clock line (shared bus)

B. Software Dependencies

Platform	Library	Version	Purpose
Raspberry Pi	MediaPipe	0.10.33	Face landmark detection
Raspberry Pi	OpenCV	Latest	Video capture and processing
Raspberry Pi	PySerial	Latest	USB serial communication
Raspberry Pi	NumPy	Latest	EAR distance calculations
Arduino	Wire.h	Built-in	I2C communication
Arduino	LiquidCrystal_I2C	Latest	LCD display control

C. Session Analysis Graphs



D. GitHub Repository

